

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1
дисциплины «Программирование на Python»

Выполнил(а):
Еремина Татьяна Евгеньевна
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Руководитель практики:
Воронкин Р.А., доцент департамента
цифровых, робототехнических систем
и электроники института
перспективной инженерии

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

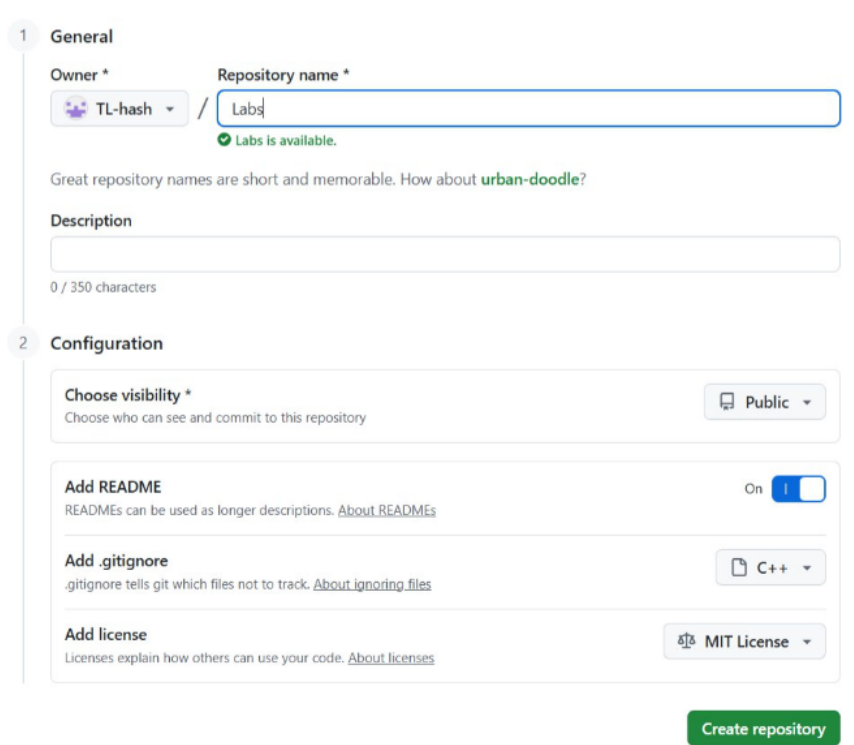
Ставрополь, 2025 г.

Тема: исследование основных возможностей Git и GitHub.

Цель: исследовать базовые возможности системы контроля версий Git и веб-версий для хостинга IT-проектов GitHub.

Практическая часть:

На основе изученного теоретического материала, создадим общедоступный репозиторий (Public), выберем лицензию Mit (Mit License) и язык программирования (C++).



The screenshot shows the GitHub 'Create new repository' form. The 'General' tab is selected, showing the repository name 'Labq' and a public visibility setting. The 'Configuration' tab is also visible, showing options to add a README, .gitignore, and a license (MIT License).

1 General

Owner * TL-hash / Repository name * Labq

Great repository names are short and memorable. How about [urban-doodle?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility * Public

Choose who can see and commit to this repository

Add README On

READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore C++

.gitignore tells git which files not to track. [About ignoring files](#)

Add license MIT License

Licenses explain how others can use your code. [About licenses](#)

Create repository

Рисунок 1. Создание и настройка репозитория в GitHub

Выполним клонирование созданного репозитория на рабочий компьютер. Для этого создадим папку на рабочем столе, в ней откроем консоль Git Bash. Чтобы клонировать репозиторий, вводим команду `git clone` и ссылку на глобальный репозиторий.

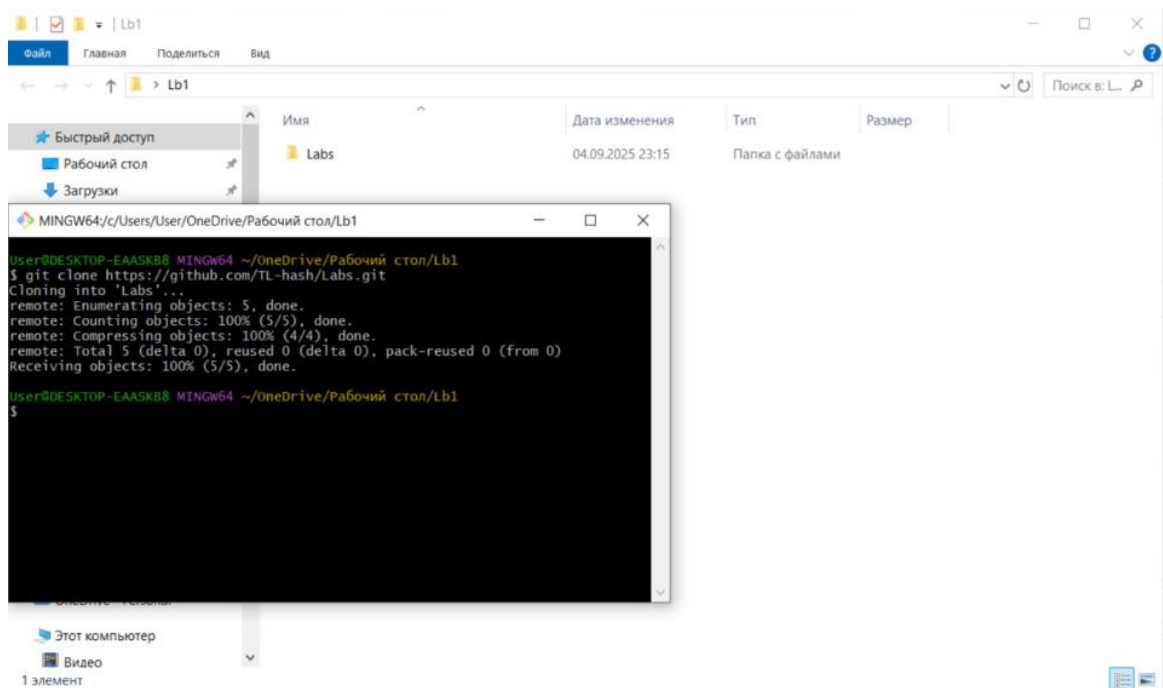


Рисунок 2. Клонирование созданного репозитория на рабочий компьютер

Перейдём в каталог репозитория, используя команду `cd` и название каталога `Labs`.

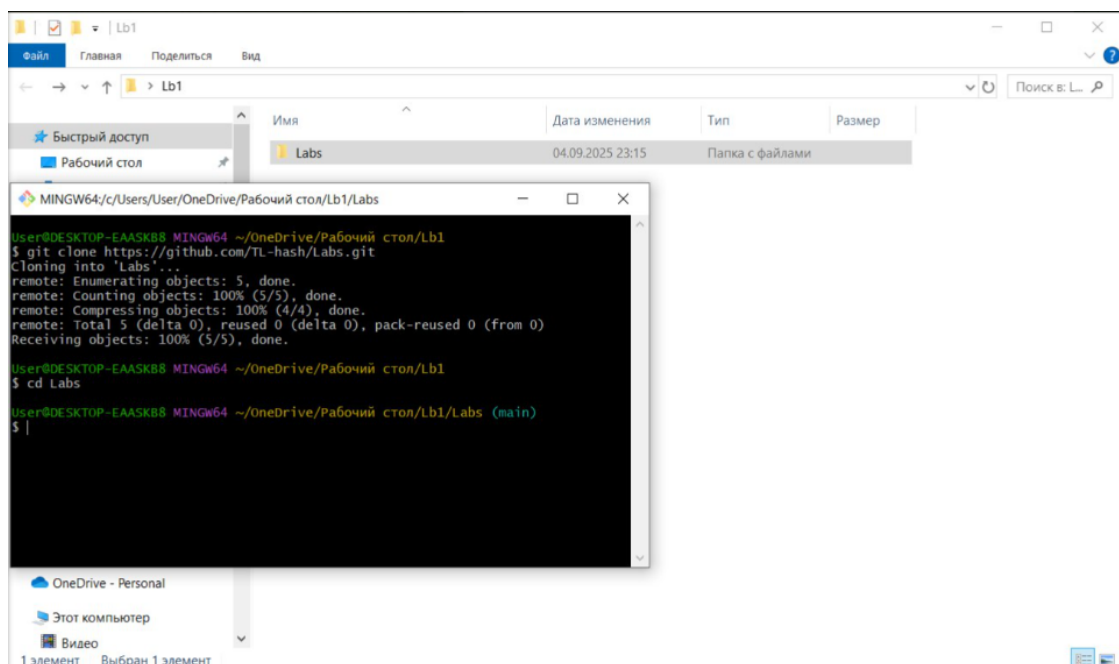


Рисунок 3. Каталог репозитория

Дополним файл `.gitignore` правилами для выбранного языка программирования и интегрированной среды разработки. Для сохранения изменений в файле, введём команду `git add .` (“.” используется для сохранения изменений во всех файлах).

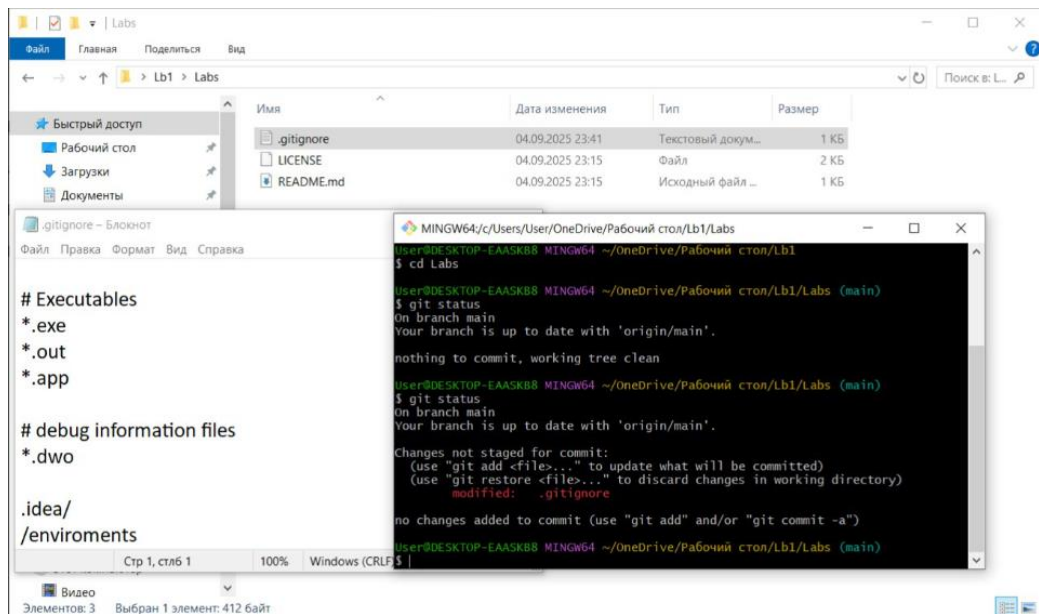


Рисунок 4. Добавление правил в файл .gitignore

В файл README.md добавим информацию о группе и ФИО, затем сохраним тем же способом.

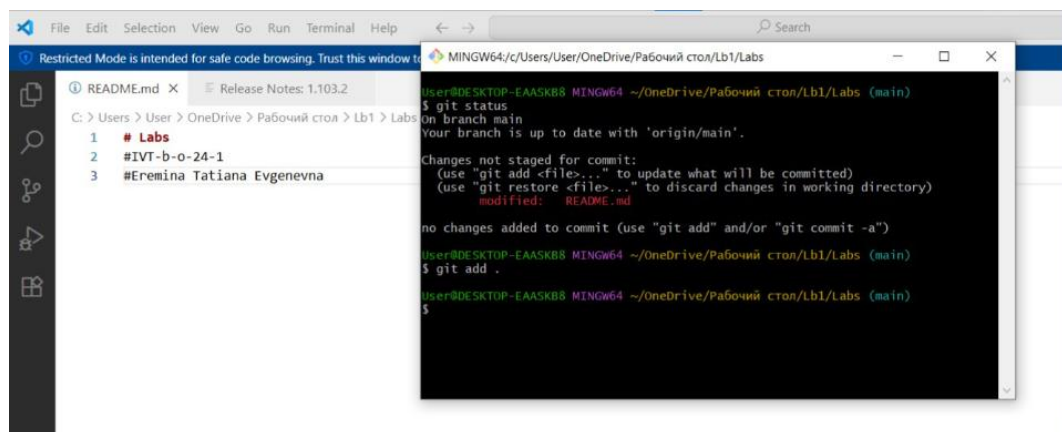


Рисунок 5. Добавление информации в файл README.md

Напишем программу простого калькулятора на языке C++. Создадим проект в Microsoft VS, разместим его в каталоге репозитория. Чтобы сохранить файл, введём команду `git add .` и создадим коммит, используя `git commit`.

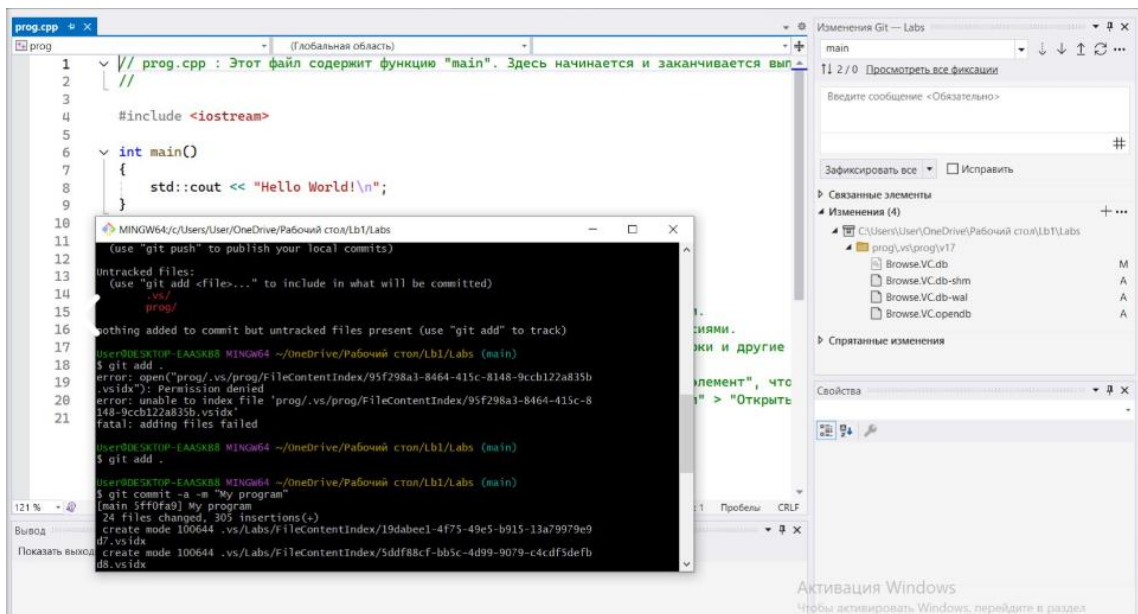


Рисунок 6. Размещение проекта с программой в каталоге репозитория

В ходе написания программы создаём коммиты.

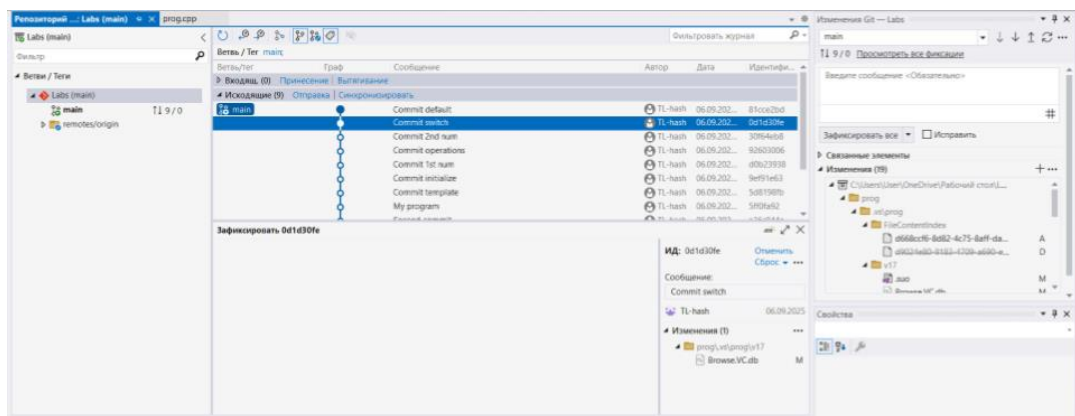


Рисунок 7. Создание коммитов

Добавляем файл README и фиксируем изменения.

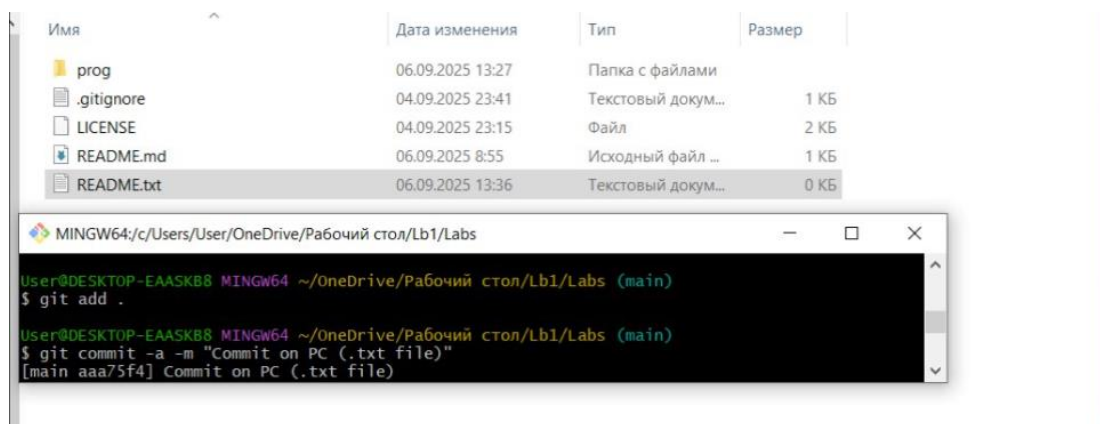


Рисунок 8. Добавление файла README

Распространим изменения в исходный репозиторий GitHub, командой git push.

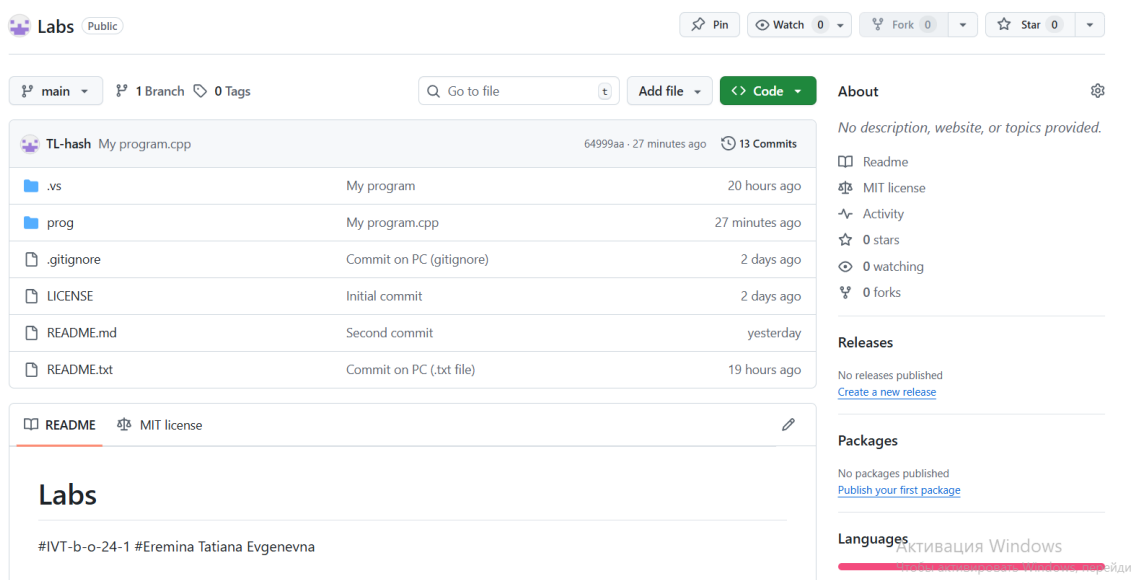


Рисунок 9. Изменения в исходном репозитории GitHub

Контрольные вопросы:

1. СКВ (система контроля версий) – это система, регистрирующая изменения в одном или нескольких файлах с тем, чтобы в дальнейшем была возможность вернуться к определённым старым версиям этих файлов.
2. Недостаток локальных и централизованных систем контроля версий – это единая точка отказа, представленная такими серверами. Если сервер выходит из строя, пользователь не может использовать контроль версий, а также есть риск потерять историю проекта.
3. Git относится к распределённым системам контроля версий.
4. Концептуальное отличие Git от других СКВ состоит в том, что другие системы хранят информацию в виде списка изменений в файлах. Git обрабатывает данные в виде набора снимков. Каждый раз, когда делается коммит, система запоминает, как выглядит каждый файл в этот момент, сохраняет ссылку на этот снимок и представляет данные, как поток снимков.
5. Целостность хранимых данных в Git обеспечивается с помощью вычисления хеш-суммы и последующего сохранения.
6. У Git есть три основных состояния, в которых могут находиться файлы: зафиксированное (committed), изменённое (modified) и подготовленное (staged). Связь этих состояний заключается в их

соответствии трём основным секциям: рабочая директория (Working Directory), область подготовленных файлов (Staging Area) и Git-директория (Git Directory). Файл в рабочей директории, который отличается от того, что находится в последнем коммите, считается `modified`. Область подготовленных файлов – это специальный файл (обычно находится в директории `.git`), который хранит информацию о том, что должно войти в следующий коммит. При создании коммита снимок файлов сохраняется в Git-директорию.

7. Профиль пользователя GitHub – это публичная страница на GitHub, где он может вносить изменения.

8. Репозитории на GitHub могут быть открытыми (`public`) и закрытыми (`private`).

9. Основные Этапы модели работы с GitHub: создание копии репозитория и клонирование на рабочий компьютер, создание ветки, внесение изменений и коммитов, синхронизация и отправка, создание `pull request`, доработка и слияние.

10. Первоначальная настройка Git после установки включает в себя команды: `git config --global user.name` и `git config --global user.email`.

11. Этапы создания репозитория в GitHub: выбор опции создания репозитория, ввод названия и описания, выбор основных настроек (лицензии, языка программирования, основных файлов `README.md`).

12. В GitHub при создании репозитория поддерживаются следующие типы лицензий: строгие лицензии с условием копилефт (GNU General Public License v3.0, GNU Affero General Public License v3.0, GNU Lesser General Public License v2.1), разрешающие лицензии (MIT License, Apache License 2.0, BSD Licenses), лицензии для публичного достояния и отказа от прав (The Unlicense, Creative Commons Zero v1.0 Universal).

13. Клонирование репозитория GitHub осуществляется с помощью URL, который необходим при вводе команды в консоли Git Bash `git clone URL`. Клонирование репозитория необходимо для работы над проектами с

открытым исходным кодом, для получения истории изменений, для создания локальной копии с возможностью синхронизации.

14. Чтобы проверить состояние локального репозитория в Git, можно использовать команду `git status`, которая покажет внесённые изменения и коммиты.

15. При добавлении/изменении файла в рабочей директории локальный репозиторий (`.git`) еще ничего не знает о этих изменениях. Они существуют только на диске. При добавлении файла под версионный контроль (`git add`) локальный репозиторий знает об этих изменениях и подготовил их для следующего коммита, это промежуточное состояние. При фиксации изменений (`git commit`) история локального репозитория обновилась, появился новый коммит, который указывает на предыдущий. Ветка теперь указывает на этот новый коммит, но на удаленном сервере (GitHub) пока ничего не изменилось. При отправке изменений на сервер (`git push`) локальный и удаленный репозитории синхронизируются.

16. На обоих рабочих компьютерах необходимо клонировать репозиторий GitHub командой `git clone URL` и перейти в каталог локального репозитория командой `cd "имя каталога"`. После того, как закончена работа на первом компьютере, проверяем состояние локального репозитория `git status`, добавляем изменения в индекс `git add .`, фиксируем изменения с комментарием `git commit -m "..."`, отправляем изменения на сервер `git push`. На втором компьютере проверяем состояние локального репозитория на наличие изменений `git status` и получаем изменения из GitHub `git pull`. После работы проделываем те же действия, что и с первым компьютером.

17. Помимо GitHub существуют такие сервисы, как GitLab, Bitbucket, Azure DevOps Services, Gitea / Forgejo, SourceHut и др. Сравним сервисы GitHub и BitBucket (таблица 1).

Таблица 1. Сравнение сервисов GitHub и Bitbucket

Критерий	GitHub	Bitbucket
Владелец	Microsoft	Atlassian

Система контроля версий	Git	Git и Mercurial (Mercurial до 2020 г.)
Интеграции	Огромный Marketplace с тысячами приложений и действий	Интеграция с Jira, Trello, Confluence, Opsgenie (стек Atlassian)
Code Review	Pull Requests	Pull Requests
Безопасность	Advanced Security (сканирование кода на уязвимости, секреты, зависимости)	Bitbucket Cloud Security (сканирование тайных ключей, 2FA, отчеты о зависимостях) на платных тарифах

18. Существуют следующие программные средства с графическим интерфейсом пользователя для работы с Git: GitHub Desktop, GitKraken, Sourcetree, Fork и др. Например, в GitHub Desktop операция клонирования происходит через опцию Clone repository и копирования ссылки. GitHub Desktop автоматически обнаруживает изменения, комментирование выполняется вводом сообщения и нажатия на кнопку Commit. После создания коммита в интерфейсе появляется кнопка Push origin для отправки изменений на сервер. Для принудительной проверки обновлений с сервера используется кнопка Fetch origin.

Вывод: были исследованы базовые возможности системы контроля версий Git и веб-версий для хостинга IT-проектов GitHub.